



Callback Functions in JavaScript

JavaScript



Contents

- Introduction
 - What is a callback function
- Ways of writing callback functions
 - Named callbacks
 - Anonymous callbacks
 - Arrow function callbacks
- Types of callbacks
 - Synchronous callbacks
 - Asynchronous callbacks
- Conclusion
 - Recap

◆ Introduction

Have you ever wondered how JavaScript knows what to do **after** completing a task like waiting for a timer, handling user clicks, or fetching data from a server? That's where **callback functions** come in. In this article, we're going to learn the essentials of callbacks in JavaScript

◆ What is a Callback?

In JavaScript, functions are first-class citizens, meaning they can be passed around like variables. A **callback** is simply a function passed as an argument to another function, which gets executed later either immediately, or after a specific event or operation.

Basic example:



```
function greet(name) {  
  console.log("Hello, " + name);  
}  
  
function runCallback(callback) {  
  const name = "Rowland";  
  callback(name); // invoke the callback function  
}  
  
runCallback(greet); // Output: Hello, Rowland
```

In the above example, the `greet` function is the callback function passed into the `runCallback` function and was executed in the second line of the `runCallback` function.

You will notice in the last line of the code, when passing in the `greet` function as an argument to `runCallback`, we did not call the function by adding parentheses to the argument.



```
runCallback(greet()); // Incorrect
runCallback(greet); // Correct
```

That is because doing so would run the `greet` function first and pass in the returned value (undefined) as the argument to `runCallback` function which is not what we want. We instead pass in a reference of the function we want to call by not adding the parentheses.

◆ Ways of Writing Callback Functions

1. Named Callbacks

A named callback is a callback function that has a name instead of being anonymous or inline.

The first example above uses a named callback (`greet`)

Benefits of using named callbacks:

- Reusability
- Readability
- Debugging
- Separation of concerns

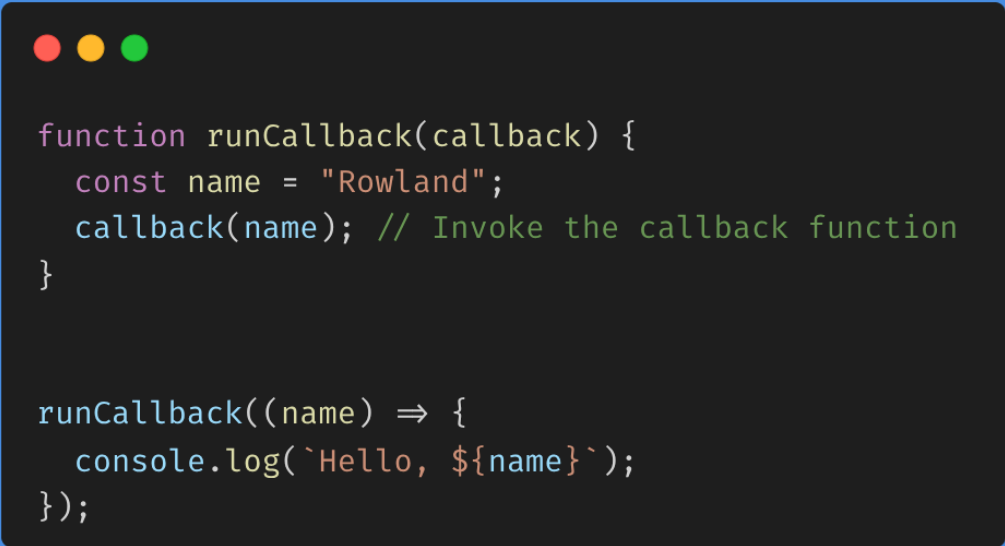
2. Anonymous Callbacks

An anonymous callback is a function without a name that is passed directly as an argument to another function — usually inline.

```
function runCallback(callback) {  
  const name = "Rowland";  
  callback(name); // invoke the callback function  
}  
  
runCallback(function(name) {  
  console.log("Hello, " + name);  
})
```

3. Arrow Function Callbacks

As the name suggests, this is simply passing in an arrow function as the callback function, this is similar to anonymous callbacks but uses modern es6 arrow function instead.



```
function runCallback(callback) {  
  const name = "Rowland";  
  callback(name); // Invoke the callback function  
}  
  
runCallback((name) => {  
  console.log(`Hello, ${name}`);  
});
```

◆ Types of Callback Functions

1. Synchronous Callbacks

A synchronous callback is a function that is executed immediately within the function it is passed to — during the same call stack execution. This means the function completes before the next line of code runs

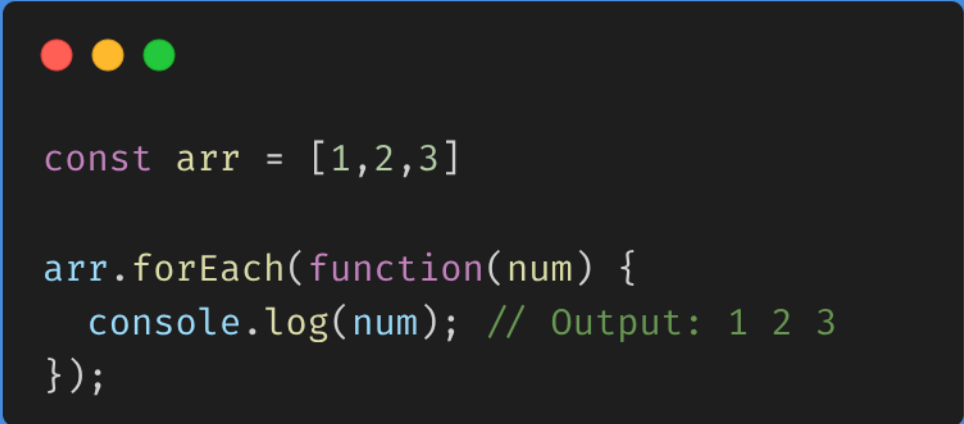
Common use cases of synchronous callbacks

- Array methods (forEach, map, filter, etc)
- Custom utility functions
- Higher-Order functions

Most array methods that accepts callback functions are of the form
`Array.methodName(callback)`

Such array methods are referred to as Higher-Order Functions (HOF) because they accept another function (callbacks) as an argument.

Examples with forEach array method

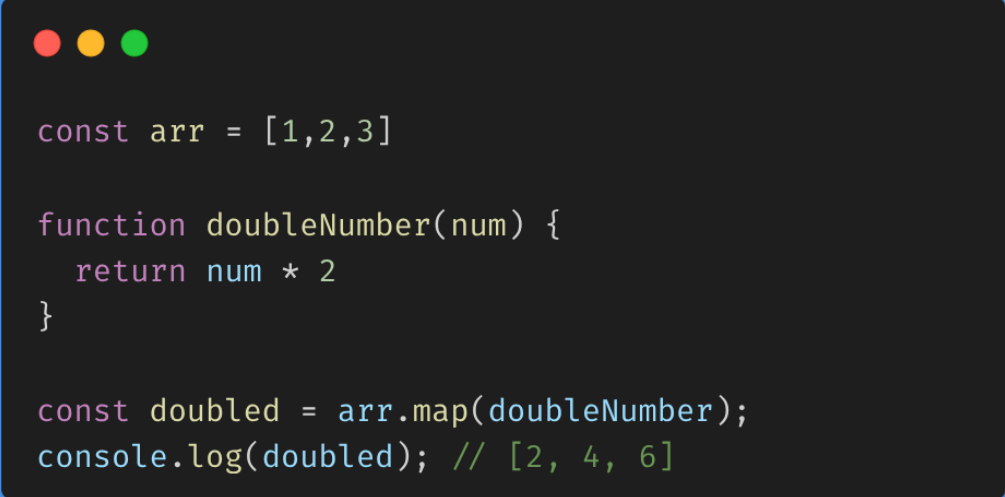


```
const arr = [1,2,3]

arr.forEach(function(num) {
  console.log(num); // Output: 1 2 3
});
```

From the above example we passed an *anonymous callback* to the `arr.forEach` method to log each of the items of the array.

Examples with map array method



```
const arr = [1,2,3]

function doubleNumber(num) {
  return num * 2
}

const doubled = arr.map(doubleNumber);
console.log(doubled); // [2, 4, 6]
```

From the above, we passed in a *named callback* to the `arr.map` method to double each item of the array.

5. Asynchronous Callbacks

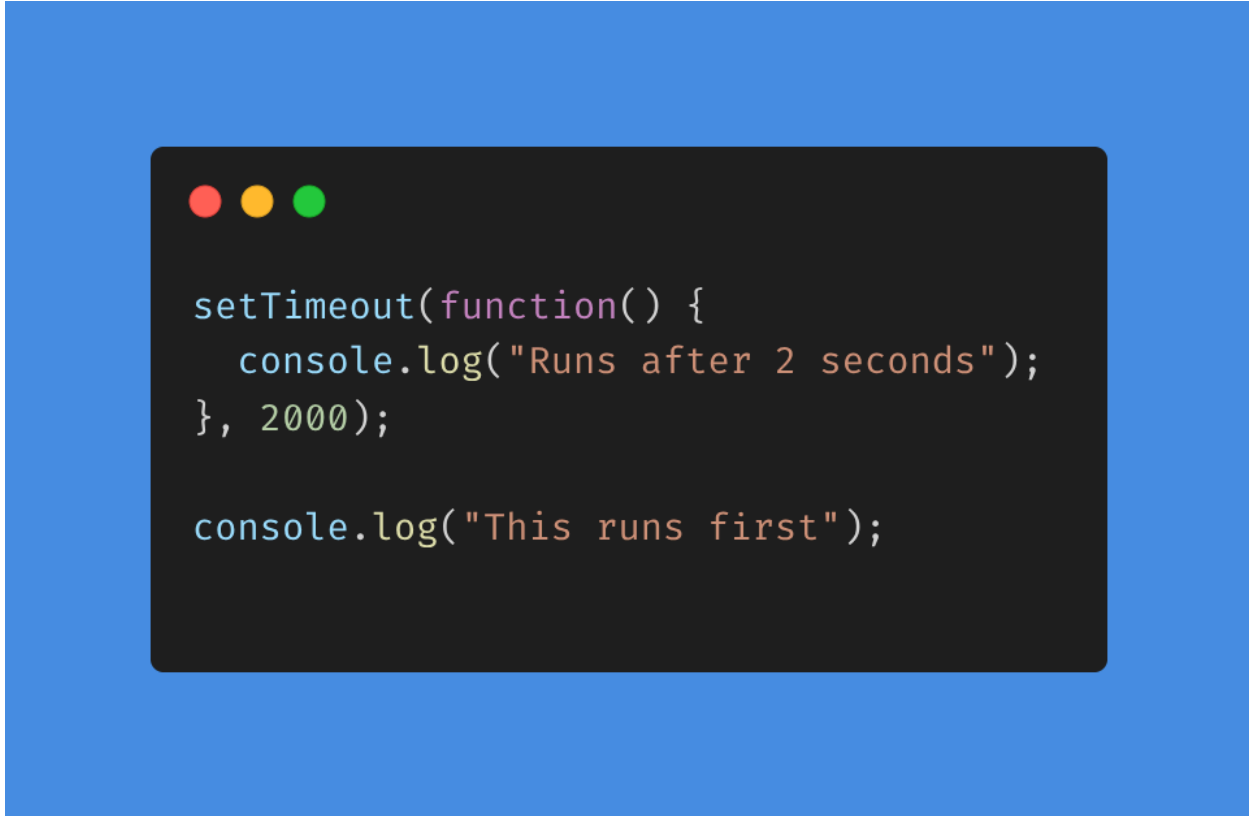
An asynchronous callback is a function passed into another function that is executed after an asynchronous operation completes, not immediately.

Common Asynchronous Operations

- setTimeout
- Event listeners
- HTTP requests

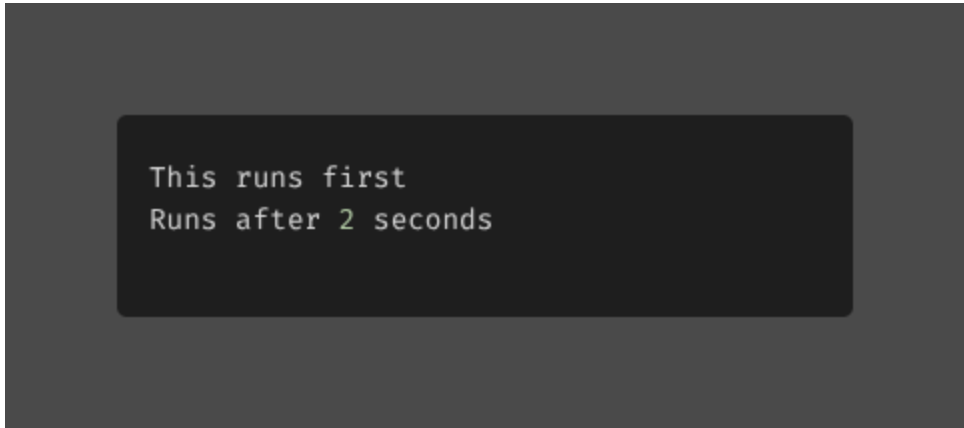
SetTimeout

The `setTimeout` function typically accepts 2 arguments, a callback function and a timer in milliseconds. The callback gets executed after the timer is up.



```
setTimeout(function() {  
  console.log("Runs after 2 seconds");  
}, 2000);  
  
console.log("This runs first");
```

From the above example, the output in the console will be:



```
This runs first  
Runs after 2 seconds
```

That is because the JavaScript engine typically does not wait for an asynchronous task to complete before moving on to the next line so while the callback function is waiting for 2 seconds to be executed the next line of code gets executed immediately and logs to the console.

Event Listeners

An **event listener** is a function that waits for a specific event (like a click, keypress, hover, etc.) and executes in response.

It has the syntax `element.addEventListener(eventType, callback);`



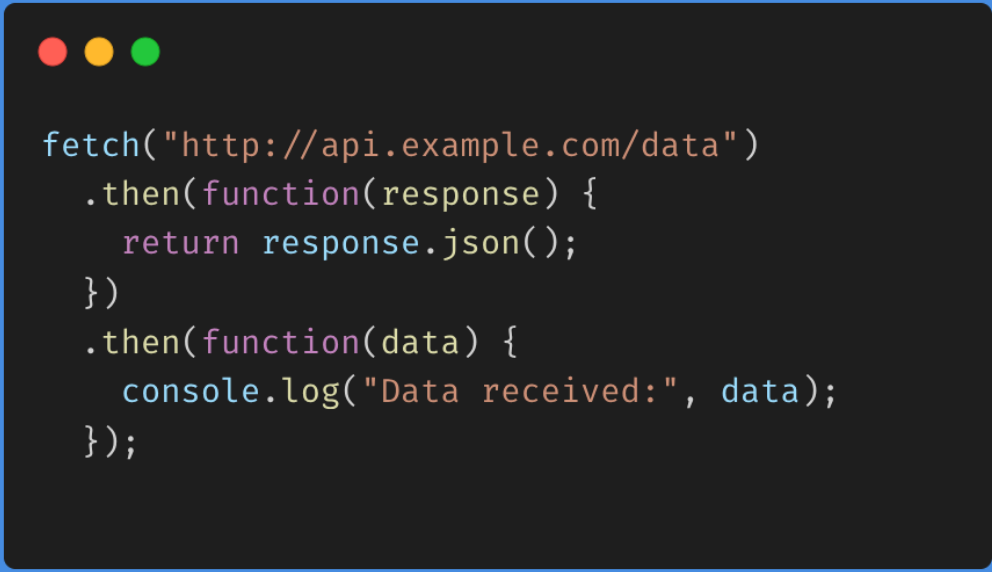
```
const btn = document.getElementById("btn")

btn.addEventListener("click", function() {
  console.log("Button was clicked!");
});
```

In the above example, the callback function gets executed only when the user clicks on the button element.

HTTP Requests

We often make http requests to an api to get data using the `fetch` function which returns a promise which we chain using the `then` function.



```
fetch("http://api.example.com/data")
  .then(function(response) {
    return response.json();
  })
  .then(function(data) {
    console.log("Data received:", data);
  });
```

In the example above, we passed in 2 anonymous callback functions to the `.then` method, first to parse the response to json and second to console log the received data. Note that we could easily have used either named or arrow callbacks if we wanted to and most people prefer to use arrow function callbacks for its simplicity and readability.



◆ Conclusion

Callback functions are foundational to understanding how JavaScript handles function execution especially when dealing with asynchronous tasks. Understanding callbacks helps you understand array methods, data fetching and event listeners better.

To recap:

- **Callbacks** are functions passed into other functions to be invoked later.
- You can define them as **named functions**, **anonymous functions**, or **arrow functions**, each with its own use cases and advantages.
- **Synchronous callbacks** run immediately and are common in array methods like `forEach`, `map`, and `filter`.
- **Asynchronous callbacks** run after an operation completes like with `setTimeout`, event listeners, or HTTP requests using `fetch`.

As you write more JavaScript, you'll find callbacks are everywhere. Grasping how and when they execute is crucial for writing effective, non-blocking, and clean JavaScript code. This knowledge also sets the foundation for deeper concepts like Promises and `async/await`.



◆ Connect with Me

If you found this helpful and want to learn more about JavaScript, web development, and other tech topics, feel free to connect with me on:

- **X (formerly Twitter):** https://x.com/rolex_devv
- **GitHub:** <https://github.com/rowleks>
- **LinkedIn:** <https://www.linkedin.com/in/rowland-momoh-b32a7a22a/>
- **Personal Portfolio:** <https://rowland-momoh.netlify.app/>

Let's keep learning and building together!